

Binary tree.

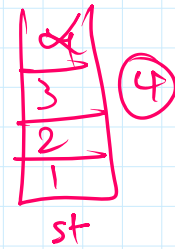
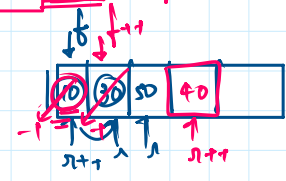
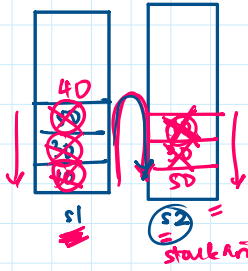
implement Queue using stacks.

implement stack using queue

* 2 stacks

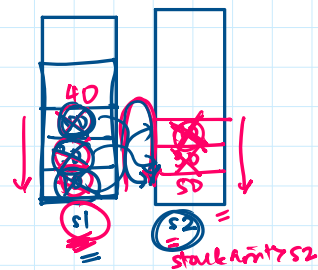
* finish functions

stack init s1, s2
push() pop() push() pop()
general Queue (implementation)

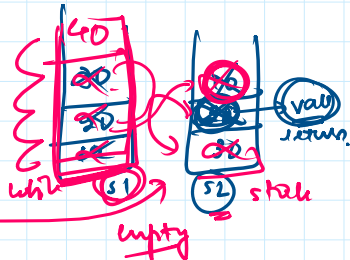


front = 1 (-1 → delete the element)
rear = n (inserting new element)

stack init s1, s2;
void push(int x) {
 s1.push(x);
}

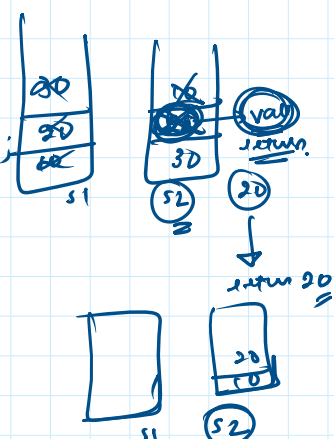


int pop() {
 if (s2.empty()) {
 while (!s1.empty()) {
 s2.push(s1.top());
 s1.pop();
 }
 int val = s2.top();
 s2.pop();
 return val;
 }
 return s2.top();
}



Stack ← flip
Queue ← peek()

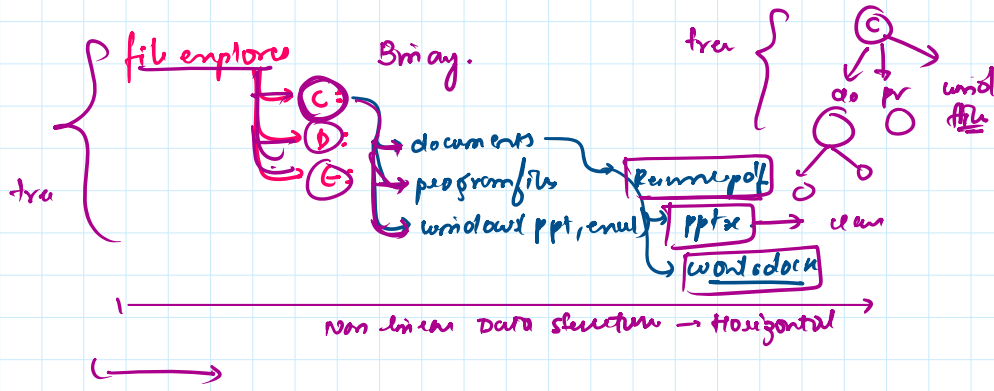
bool empty() {
 return s1.empty() && s2.empty();
}



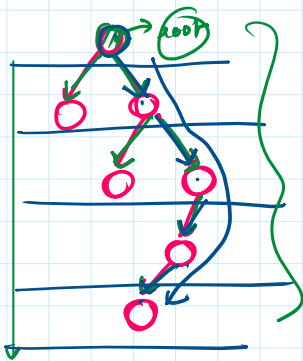
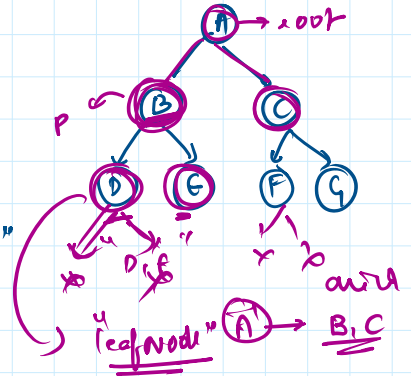
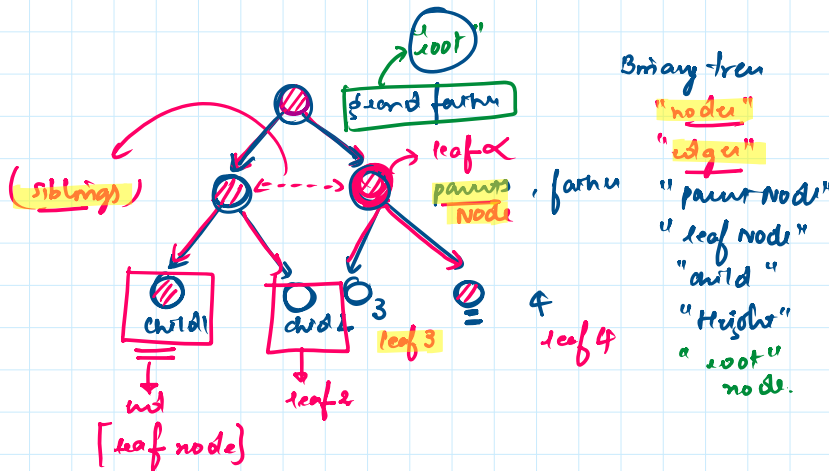
bool empty() {
 return s1.empty() && s2.empty();
}

empty Queue empty

Assumption $\rightarrow$ PC $\rightarrow$ file manager / file explorer / my computer



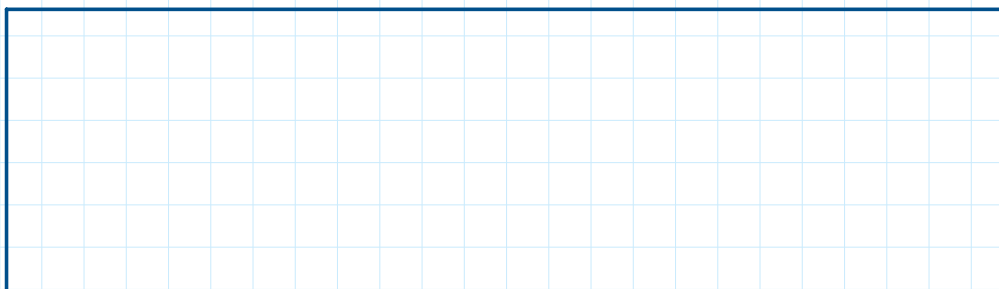
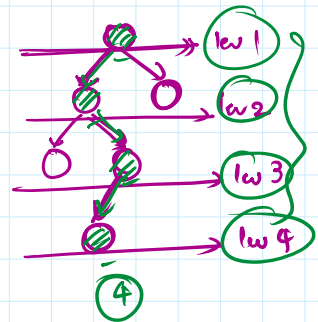
Height



Google maps

$\rightarrow$ Height of the tree $\rightarrow$ longest distance
"number of nodes"
"count of height" = 5
"root $\rightarrow$ leaf node"
"leaf node"
"wild"
"Height"
"root node"

ht = 1

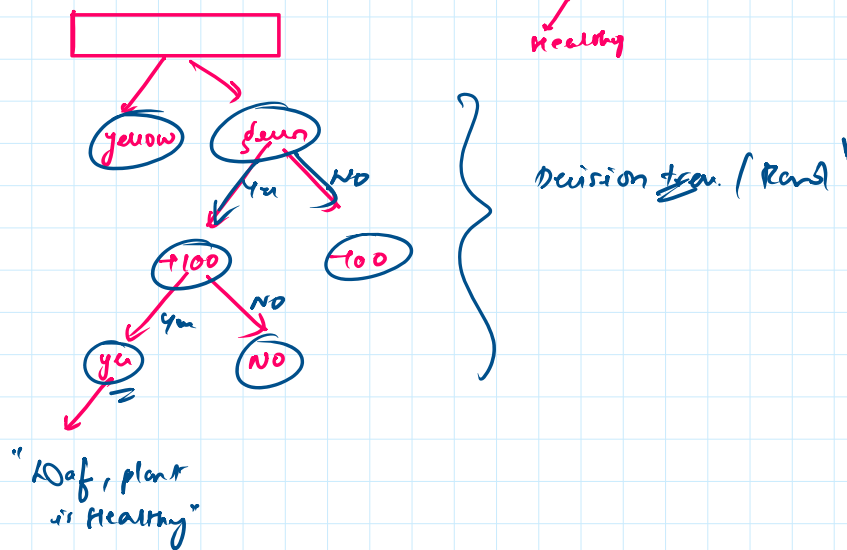
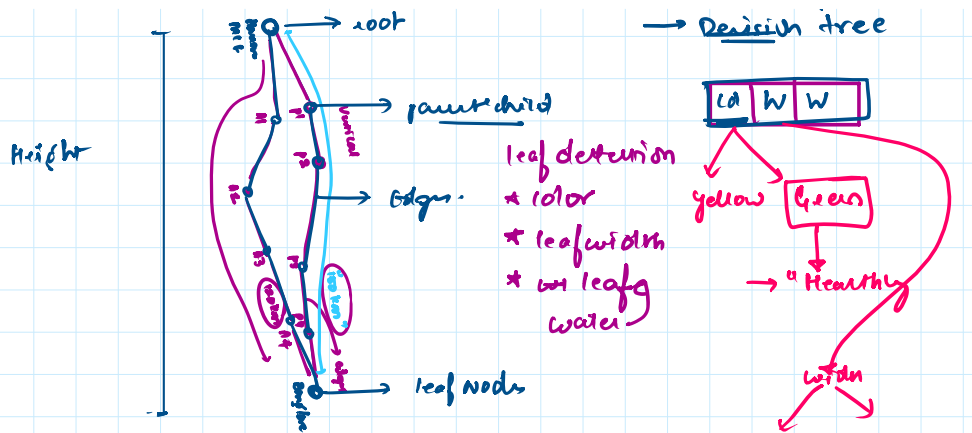


PC Binary tree

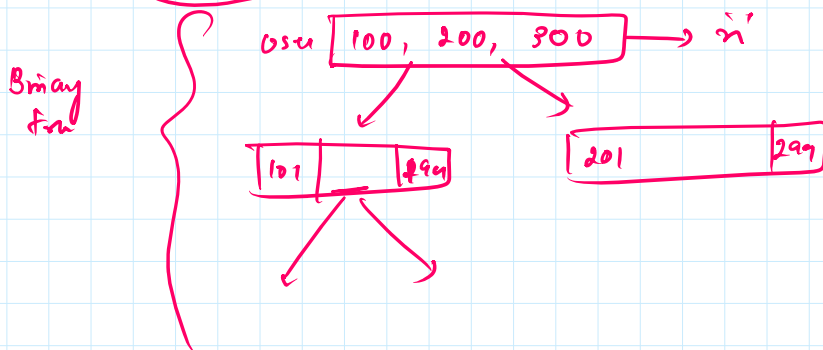
maine

$\rightarrow$ Random forest
 $\rightarrow$ Decision tree

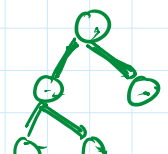
1d 1w 1w



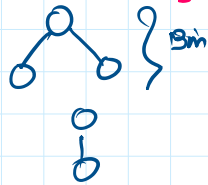
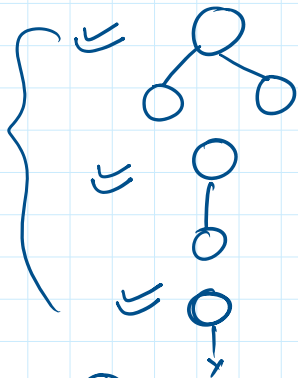
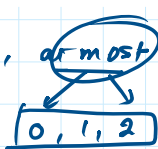
DBMS → B+ tree / B tree



* definition: Non linear D.S., at most 2 nodes children



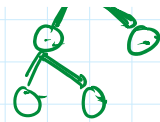
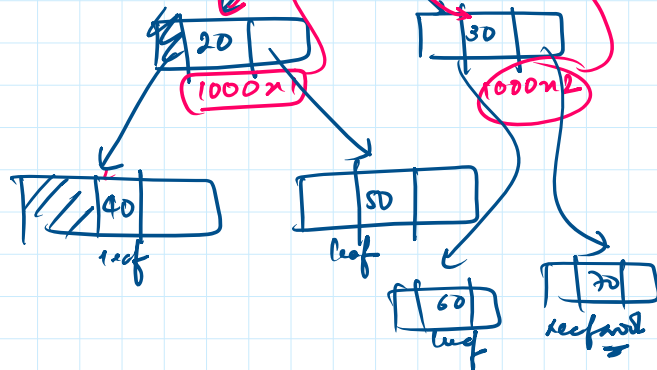
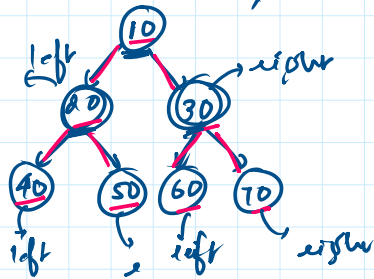
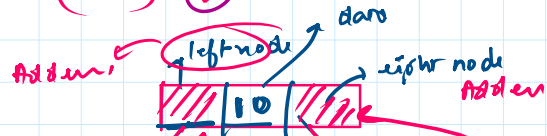
* definition: Non linear D.S, at most 2 nodes children



nodes: (n) → Question
edges: $(n-1)$

$(n-1) = \text{nodes of edges}$

$(7-1) = 6$



$5 - 1 = 4$
 $n - 1 = 4$
not edge

$7 - 1 = 6$ edge